Programa:	CubisGold	
Protección:	Armadillo + Límite 5 ejecuciones	
Objetivos:	Obtener un serial valido	
URL descarga:	www.freshgames.com	
Dificultad:	Fácil	
Herramientas:	Ollydbg 1.10 + Import Reconstructor + LordPE + PEditor + PEiD	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 12-05-2005

Hoooola a tod@s..!!

hoy estudiaremos un poco al Packer ***Armadillo 3.00a - 3.61 -> Silicon Realms Toolworks*** como nos dice ***PEiD***, como siempre hacemos, esta vez la víctima es un juego de la casa ***Fresh Games*** y se llama ***CubisGold***, no he jugado sólo lo he mirado por encima hay que ir poniendo cubiletes en fila o algo así ¿? pero nosotros venimos a jugar de otra manera, como antes dije pasemos al primer paso:

Armadillo 3.00a - 3.61 -> Silicon Realms Toolworks

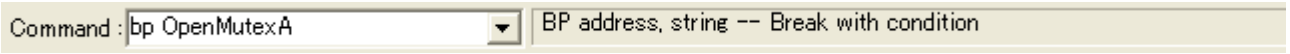
lo dicho hasta ahora nada nuevo, vale cargamos el programa en ***Olly***, tildamos de momento todas las excepciones para que ***Olly*** no pare en ninguna e iremos más rápido, queremos saber si tiene o no ***antidebug***, hecho esto damos ***RUN*** (F9) y el programa carga, aparece esta nag en pantalla:

nos queda un sólo intento según allí dice en rojo. de momento pulsamos el botón "***Try It***" y vemos que el programa carga, miremos cuantos procesos se crearon desde el ***LordPE*** por ejemplo:

Path	PID	ImageBase	ImageSize
c:\ollydbg 1.10\parcheado 4.exe	0000018C	00400000	00153000
c:\archivos de programa\freshgames\cubis g...	0000013C	00400000	00916000
c:\archivos de programa\freshgames\cubis g...	00000184	00400000	00916000
c:\tools\lordpe\lordpe.exe	000003C0	00400000	00036000

Vaya, se crea un segundo proceso, serán ***Nanomites***? quizá ***CopyMem***? veremos más adelante, lo que yo suelo hacer en estos casos es aprovecharme del uso de la ***API OpenMutexA*** para evitar que se cree el segundo proceso, esto lo aprendí hace un tiempecillo atrás de un pedazo de ***script*** que se curraba un ruso del que no sé su nombre sino de veras que lo ponía porque era buenísimo, como a mi no me funcionaba investigué las llamadas a esta ***API***, y al final encontré que en la mayoría de ***Armadillos*** la segunda es la que nos vale cambiando el valor retorno que

trae ***EAX*** evitamos que se cree el proceso hijo, he de comentar además que también encontré un Armadillo en el que el retorno (valor de ***EAX***) se debía cambiar en la primera llamada, de todos modos es muy sencillo porque a la salida de la llamada "buena" donde debemos modificar ***EAX*** siempre vemos referencias a ***IsDebuggerPresent*** y a ***GetProcAddress*** un poco antes, pongo la imagen de este caso concreto para que se vea mejor:



Primero ponemos el ***BP*** como se ve en la imagen, lo escribiremos tal cual respetando mayúsculas y minúsculas sino no lo reconocerá y no quedará puesto, cuando lo tengamos escrito como en la imagen pulsamos ***Enter*** y si no dice nada ya estará, si se queja repasaremos la ortografía. una vez lo tengamos pues nada damos ***RUN*** y a correr... romperá el ***BP*** por primera vez, no le hacemos caso, la segunda vez que para pulsamos ***Ctrl+F9*** y salimos aquí:

```
009D03C7   52              PUSH EDX
009D03C8   6A 00           PUSH 0
009D03CA   68 01001F00     PUSH 1F0001
009D03CF   FF15 54B09F00   CALL DWORD PTR DS:[<&KERNEL32.OpenMutex  kernel32.OpenMutexA
009D03D5   85C0            TEST EAX,EAX
009D03D7 v 0F85 6D020000   JNZ CubisGol.009D064A
009D03DD   6A 01           PUSH 1
009D03DF   FF15 C4B09F00   CALL DWORD PTR DS:[<&KERNEL32.GetCurren  kernel32.GetCurrentThread
009D03E5   50              PUSH EAX
009D03E6   FF15 5CB19F00   CALL DWORD PTR DS:[<&KERNEL32.SetThread  kernel32.SetThreadPriority
009D03EC   C685 54F9FFFF   MOV BYTE PTR SS:[EBP-6AC],0
009D03F3   68 E8B49F00     PUSH CubisGol.009FB4E8                  ASCII "Kernel32"
009D03F8   FF15 6CB19F00   CALL DWORD PTR DS:[<&KERNEL32.LoadLibra  kernel32.LoadLibraryA
009D03FE   8985 50F9FFFF   MOV DWORD PTR SS:[EBP-6B0],EAX
009D0404   83BD 50F9FFFF   CMP DWORD PTR SS:[EBP-6B0],0
009D040B v 74 32           JE SHORT CubisGol.009D043F
009D040D   68 D4B49F00     PUSH CubisGol.009FB4D4                  ASCII "IsDebuggerPresent"
009D0412   8B85 50F9FFFF   MOV EAX,DWORD PTR SS:[EBP-6B0]
009D0418   50              PUSH EAX
009D0419   FF15 74B19F00   CALL DWORD PTR DS:[<&KERNEL32.GetProcAd  kernel32.GetProcAddress
009D041F   8985 B8F8FFFF   MOV DWORD PTR SS:[EBP-748],EAX
009D0425   83BD B8F8FFFF   CMP DWORD PTR SS:[EBP-748],0
009D042C v 74 11           JE SHORT CubisGol.009D043F
009D042E   FF95 B8F8FFFF   CALL DWORD PTR SS:[EBP-748]
009D0434   85C0            TEST EAX,EAX
009D0436 v 74 07           JE SHORT CubisGol.009D043F
009D0438   C685 54F9FFFF   MOV BYTE PTR SS:[EBP-6AC],1
009D043F   C785 BCF8FFFF   MOV DWORD PTR SS:[EBP-744],94
```

Ahí están las referencias a ***IsDebugger***.. etc., como antes comenté, aquí es donde debemos actuar. simplemente cambiamos el valor de EAX a uno cualquiera mientras sea diferente de 0g, yo pongo ***EAX==1*** por ejemplo, de esta manera saltará en :9D03D7 y no creará el segundo proceso, ahora pondremos un ***BPM*** en la primera sección (sin contar el ***PE header***) para localizar el OEP, quitamos los ***BP's*** que tengamos puestos y damos RUN, aparece la ***nagscreen*** que nos dice que nos queda 1 intento ya que antes no lo gastamos porque reiniciamos Olly, aceptamos picando en "***Try It***!" y al momento romperemos en el ***OEP=43a6d9***, como se ve en la imagen:

```
0043A6D9   55              PUSH EBP
0043A6DA   8BEC            MOV EBP,ESP
0043A6DC   6A FF           PUSH -1
0043A6DE   68 E0804400     PUSH CubisGol.004480E0
0043A6E3   68 94F24300     PUSH CubisGol.0043F294
0043A6E8   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0043A6EE   50              PUSH EAX
0043A6EF   64:8925 000000  MOV DWORD PTR FS:[0],ESP
0043A6F6   83EC 58         SUB ESP,58
0043A6F9   53              PUSH EBX
0043A6FA   56              PUSH ESI
0043A6FB   57              PUSH EDI
0043A6FC   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
0043A6FF   FF15 28514400   CALL DWORD PTR DS:[445128]
0043A705   33D2            XOR EDX,EDX
0043A707   8AD4            MOV DL,AH
0043A709   8915 2C949C00   MOV DWORD PTR DS:[9C942C],EDX
0043A70F   8BC8            MOV ECX,EAX
```

OEP==43A6D9

API redirigida..!!

Ya tenemos el primer dato importante que nos anotamos claro está.

<u>OEP</u>: 43A6D9

A continuación buscamos la ***IAT*** que deberemos reconstruir ya que está algo dañada, algunos saltos a la ***API*** han sido suplantados por saltos a direcciones del código de Armadillo donde o bien emula la ***API*** en cuestión o bien te envía a ella unas líneas después del inicio para dificultar la labor. Aprovechando que debajo del ***OEP*** hay un ***CALL*** a la ***IAT*** picamos con el botón derecho sobre él y elegimos "***Follow in Dump->Memory Address***" desde aquí buscamos el inicio de la tabla, en la imagen se puede ver:

```
Address   Hex dump                                          ASCII
00445000  10 24 DA 77 9A 22 DA 77 99 4E DA 77 E5 AA 30 01   ►$ rwÚ"rwÖNrwõ¬00
00445010  FC 78 30 01 FD CD 01 51 AB 79 30 01 CC 16 0A 51   ²x00²=0Q½y00╠▬◙Q
00445020  E0 79 30 01 9F 86 C4 77 5B 58 C4 77 07 36 C4 77   ßy00ƒß-w[X-w•6-w
00445030  E0 20 C4 77 C7 5A C4 77 0E 6C C4 77 78 23 C4 77   ß -wÄZ-w♫l-wx#-w
00445040  1A 1E C4 77 71 4B C4 77 16 66 C4 77 8E 21 C4 77   →▲-wqK-w▬f-wÄ↑-w
00445050  D1 36 C4 77 98 1B C4 77 6E 3C C4 77 3D 51 C4 77   Ð6-wÿ←wn<-w=Q-w
```

Como coincide con el inicio de la sección no hay problema en encontrar el inicio, lo anotamos también:

Inicio IAT: 445000

Hacemos lo mismo pero ahora hacia abajo pues buscamos no el inicio sino el final, se encuentra fácilmente porque los valores pasan de ser uniformes a no tener nada que ver entre sí, además no apuntan a ***DLL's*** ni nada:

```
Address   Hex dump                                          ASCII
00445260  AB 79 30 01 BA FD 1E 76 08 C3 1E 76 1F 79 30 01  ½y00╢²▲v◘├▲v▼y00
00445270  62 8B B0 76 D8 1A B0 76 F2 4F B1 76 A4 79 30 01  bï╗vï→╬v=O▒vñy00
00445280  DA 41 A3 71 36 18 A3 71 E0 79 30 01 00 00 00 00  ┌Aúq6↑úqóy00....
00445290  44 17 41 54 FB 21 09 40 00 00 00 40 33 33 D3 3F  D↨AT¹!.@...@33╙?
004452A0  00 00 34 43 00 00 B4 42 61 0B 36 BC 00 00 7F 43  ..4C..┤Ba♂6╝..⌂C
004452B0  00 00 80 3F 00 00 00 00 00 00 00 00 00 00 32 40  ..Ç?..........2@
```

Parece que las últimas entradas son ***[445280]*** y ***[445284]***, por seguridad le daremos un margen:

Final IAT: 445290

Así no habrá problemas con el ***Import***, los que sobre le hacemos "***Cut Thunk***" y no habrá problema, lo siguiente que haremos será calcular su longitud, restamos ambos valores (final - inicio) y lo obtendremos rápidamente.

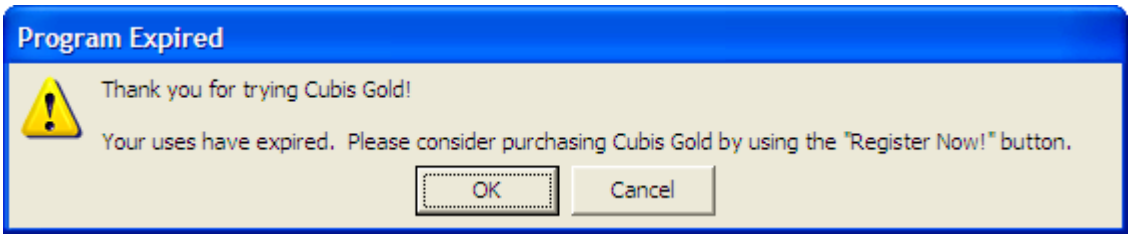
Longitud IAT: 290

Pues ya tenemos el OPE y la IAT bien delimitada, todavía nos queda recuperar los punteros redirigidos, yo siempre me fijo en el primero de ellos:

```
Address   Hex dump                                          ASCII
00445000  10 24 DA 77 9A 22 DA 77 99 4E DA 77 E5 AA 30 01  ►$┌wÜ"┌wÖN┌wÜ¬00
00445010  FC 78 30 01 FD CD 01 51 AB 79 30 01 CC 16 0A 51  ²x00²═00½y00╠▬◙Q
00445020  E0 79 30 01 9F 86 C4 77 5B 58 C4 77 07 36 C4 77  óy00ƒß─w[X─w•6─w
00445030  E0 20 C4 77 C7 5A C4 77 0E 6C C4 77 78 23 C4 77  ó ─wÄZ─w♫l─wx#─w
```

en este caso ***[44500C]==0130AAE5***, a partir de este me fijaré donde lo rellena poniéndole un ***BPHWrite->Dword***, cuando veamos que escribe el valor final estaremos en el ***loop*** donde va rellenando la ***IAT***:

pero esta vez no llegamos al ***OEP*** pues no nos dimos ni cuenta de que el programa ha expirado ¿(ahora encima a lidiar con el ***Trial***, nos dimos de morros con este mensaje:



Si queremos seguir investigando deberemos evitar esto, por suerte tenemos a ***RegMon*** en nuestras filas, jeje.. un simple vistazo con él para ver las más que sospechosas claves del registro de ***Window$:***

HKLM\Software\The Silicon Realms Toolworks\Armadillo /*un poco cantona esta*/
HKLM\Software\Licenses

Que es donde tiene el ***Arma*** sus ***flags*** que le indican el estado del programa, no vale de nada borrar estas claves sin antes eliminar del temporal el archivo "***B34A7CD6.TMP***" (o como se llame) Borrando las dos claves y este archivo chivato dispondremos de '5' usos más eh! Cuidado no nos vayamos a enganchar al juego con 5 míseras partidas, jooooder es para alucinar, a lo que vamos a darle caña al Armadillo ;DDDD

Al final solventamos este escollo, por fin sí reiniciamos ***Olly*** y nos centramos en el tema de la ***IAT***, ponemos el ***BPHWrite*** en la primera entrada que vemos mala ***[44500C]*** cuyo valor final en mi máquina era ***0130AAE5***, debemos encontrar el punto exacto donde escribe este valor. No olvidar cada vez que reiniciemos ***Olly*** de poner el ***BP OpenMutexA*** y modificar ***EAX*** en la segunda llamada a la ***API***, sin esto no haremos nada, necesitamos que no cree el segundo proceso, importante esto! Ponemos el ***BP*** en la entrada y para aquí:

```
013241D3   F3:A5          REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
013241D5   FF2495 E842320 JMP DWORD PTR DS:[EDX*4+13242E8]
013241DC   8BC7           MOV EAX,EDI
013241DE   BA 03000000    MOV EDX,3
013241E3   83E9 04        SUB ECX,4
013241E6 v 72 0C          JB SHORT 013241F4
```

Pero no hay nada, todavía no ha escrito el valor final, damos ***RUN*** nuevamente y para aquí:

```
0131EBC4   83C4 14          ADD ESP,14
0131EBC7   33C0             XOR EAX,EAX
0131EBC9   E9 560C0000      JMP 0131F824
0131EBCE   8B85 98E9FFFF    MOV EAX,DWORD PTR SS:[EBP-1668]
0131EBD4   8B8D 78E7FFFF    MOV ECX,DWORD PTR SS:[EBP-1888]
0131EBDA   8908             MOV DWORD PTR DS:[EAX],ECX      <- Aqui guarda!!
0131EBDC   8B85 98E9FFFF    MOV EAX,DWORD PTR SS:[EBP-1668]     CubisGol.0044500C
0131EBE2   83C0 04          ADD EAX,4
0131EBE5   8985 98E9FFFF    MOV DWORD PTR SS:[EBP-1668],EAX
0131EBEB   E9 7CFCFFFF      JMP 0131E86C
0131EBF0   0FB685 A4E9FFFF  MOVZX EAX,BYTE PTR SS:[EBP-165C]
0131EBF7   85C0             TEST EAX,EAX
0131EBF9   74 7F            JE SHORT 0131EC7A
0131EBFB   6A 00            PUSH 0
```

Esto sí es lo que buscamos, he seleccionado el lugar exacto donde guarda ese ***MOV [EAX],ECX*** un poco más abajo el ***JMP*** que nos envía al inicio de la rutina:

```
0131E86C   6A 01            PUSH 1                          <- Inicio rutina...
0131E86E   58               POP EAX
0131E86F   85C0             TEST EAX,EAX
0131E871   0F84 79030000    JE 0131EBF0
0131E877   66:83A5 7CE7FFF  AND WORD PTR SS:[EBP-1884],0
0131E87F   83A5 74E7FFFF    AND DWORD PTR SS:[EBP-188C],0
0131E886   83A5 78E7FFFF    AND DWORD PTR SS:[EBP-1888],0
0131E88D   8B85 88ECFFFF    MOV EAX,DWORD PTR SS:[EBP-1378]
```

nuestro próximo objetivo es buscar el salto mágico, primero de todo yo suelo buscar el ***CALL*** que pone en ***EAX*** la ***Api*** correcta y a partir de aquí investigo los saltos que eviten pasar por esta llamada, si no lo hay entro dentro e investigo pues a veces está por ahí y hay veces que retorna uno bueno y otras uno malo, en este caso es en este:

```
0131EB12   FFB5 B0D3FFFF    PUSH DWORD PTR SS:[EBP-2C50]
0131EB18   FFB5 B0E9FFFF    PUSH DWORD PTR SS:[EBP-1650]
0131EB1E   E8 D483FEFF      CALL 01306EF7                   Pone la API en EAX..!!
0131EB23   59               POP ECX                         WSOCK32.#1139
0131EB24   59               POP ECX
0131EB25   8985 78E7FFFF    MOV DWORD PTR SS:[EBP-1888],EAX
```

Allí pone en ***EAX*** la ***Api*** correcta, si miramos en la ventana de registros vemos:

```
Registers (FPU)
EAX 71A341DA WS2_32.WSAStartup
ECX 0012ED3C
EDX 77FB306C ntdll.77FB306C
```

En este casó tocó una de la librería ***WS2_32.dll*** y la ***Api*** corresponde a ***WSAStartup***, ahora tracearemos en busca del salto mágico, conociendo la dirección del ***CALL*** anterior podemos encontrarlo fácilmente, si miramos un poco más arriba del ***CALL*** del que estamos hablando vemos esto:

```
0131EAE3   83BD 78E7FFFF    CMP DWORD PTR SS:[EBP-1888],0
0131EAEA   75 3F            JNZ SHORT 0131EB2B
0131EAEC   0FB785 7CE7FFFF  MOVZX EAX,WORD PTR SS:[EBP-1884]
0131EAF3   85C0             TEST EAX,EAX
0131EAF5   74 0F            JE SHORT 0131EB06
0131EAF7   0FB785 7CE7FFFF  MOVZX EAX,WORD PTR SS:[EBP-1884]
0131EAFE   8985 B0D3FFFF    MOV DWORD PTR SS:[EBP-2C50],EAX
0131EB04   EB 0C            JMP SHORT 0131EB12
0131EB06   8B85 74E7FFFF    MOV EAX,DWORD PTR SS:[EBP-188C]
0131EB0C   8985 B0D3FFFF    MOV DWORD PTR SS:[EBP-2C50],EAX
0131EB12   FFB5 B0D3FFFF    PUSH DWORD PTR SS:[EBP-2C50]
0131EB18   FFB5 B0E9FFFF    PUSH DWORD PTR SS:[EBP-1650]
0131EB1E   E8 D483FEFF      CALL 01306EF7                   Pone la API en EAX..!!
0131EB23   59               POP ECX
0131EB24   59               POP ECX
0131EB25   8985 78E7FFFF    MOV DWORD PTR SS:[EBP-1888],EAX
0131EB2B   83BD 78E7FFFF    CMP DWORD PTR SS:[EBP-1888],0
```

Un salto en ***:0131EAEA*** que no va a saltar, entonces pasa por el ***CALL 01306EF7*** y...

```
0131EBCE   8B85 98E9FFFF    MOV EAX,DWORD PTR SS:[EBP-1668]
0131EBD4   8B8D 78E7FFFF    MOV ECX,DWORD PTR SS:[EBP-1888]
0131EBDA   8908             MOV DWORD PTR DS:[EAX],ECX      kernel32.SetStdHandle
0131EBDC   8B85 98E9FFFF    MOV EAX,DWORD PTR SS:[EBP-1668]
0131EBE2   83C0 04          ADD EAX,4
0131EBE5   8985 98E9FFFF    MOV DWORD PTR SS:[EBP-1668],EAX
0131EBEB   E9 7CFCFFFF      JMP 0131E86C
```

Guarda una ***Api*** buena, sigamos hasta que veamos que va a saltar como ahora:

```
0131EAE3   83BD 78E7FFFF    CMP DWORD PTR SS:[EBP-1888],0
0131EAEA   75 3F            JNZ SHORT 0131EB2B
0131EAEC   0FB785 7CE7FFFF  MOVZX EAX,WORD PTR SS:[EBP-1884]
0131EAF3   85C0             TEST EAX,EAX
0131EAF5   74 0F            JE SHORT 0131EB06
0131EAF7   0FB785 7CE7FFFF  MOVZX EAX,WORD PTR SS:[EBP-1884]
0131EAFE   8985 B0D3FFFF    MOV DWORD PTR SS:[EBP-2C50],EAX
0131EB04   EB 0C            JMP SHORT 0131EB12
0131EB06   8B85 74E7FFFF    MOV EAX,DWORD PTR SS:[EBP-188C]
0131EB0C   8985 B0D3FFFF    MOV DWORD PTR SS:[EBP-2C50],EAX
0131EB12   FFB5 B0D3FFFF    PUSH DWORD PTR SS:[EBP-2C50]
0131EB18   FFB5 B0E9FFFF    PUSH DWORD PTR SS:[EBP-1650]
0131EB1E   E8 D483FEFF      CALL 01306EF7                   Pone la API en EAX..!!
0131EB23   59               POP ECX
0131EB24   59               POP ECX
0131EB25   8985 78E7FFFF    MOV DWORD PTR SS:[EBP-1888],EAX
0131EB2B   83BD 78E7FFFF    CMP DWORD PTR SS:[EBP-1888],0
```

Sigamos hasta ver lo que guarda:

```
0131EBCE   8B85 98E9FFFF  MOV EAX,DWORD PTR SS:[EBP-1668]
0131EBD4   8B8D 78E7FFFF  MOV ECX,DWORD PTR SS:[EBP-1888]
0131EBDA   8908           MOV DWORD PTR DS:[EAX],ECX
0131EBDC   8B85 98E9FFFF  MOV EAX,DWORD PTR SS:[EBP-1668]
0131EBE2   83C0 04        ADD EAX,4
0131EBE5   8985 98E9FFFF  MOV DWORD PTR SS:[EBP-1668],EAX
0131EBEB ^ E9 7CFCFFFF    JMP 0131E86C
```

jejeje... una ***MALA***! si miramos el valor de ***ECX*** vemos:

```
Registers (FPU)
EAX 004450C0 CubisGol.004450C0
ECX 01308195
EDX 0012D368 ASCII "FreeLibrary"
EBX 00000000
```

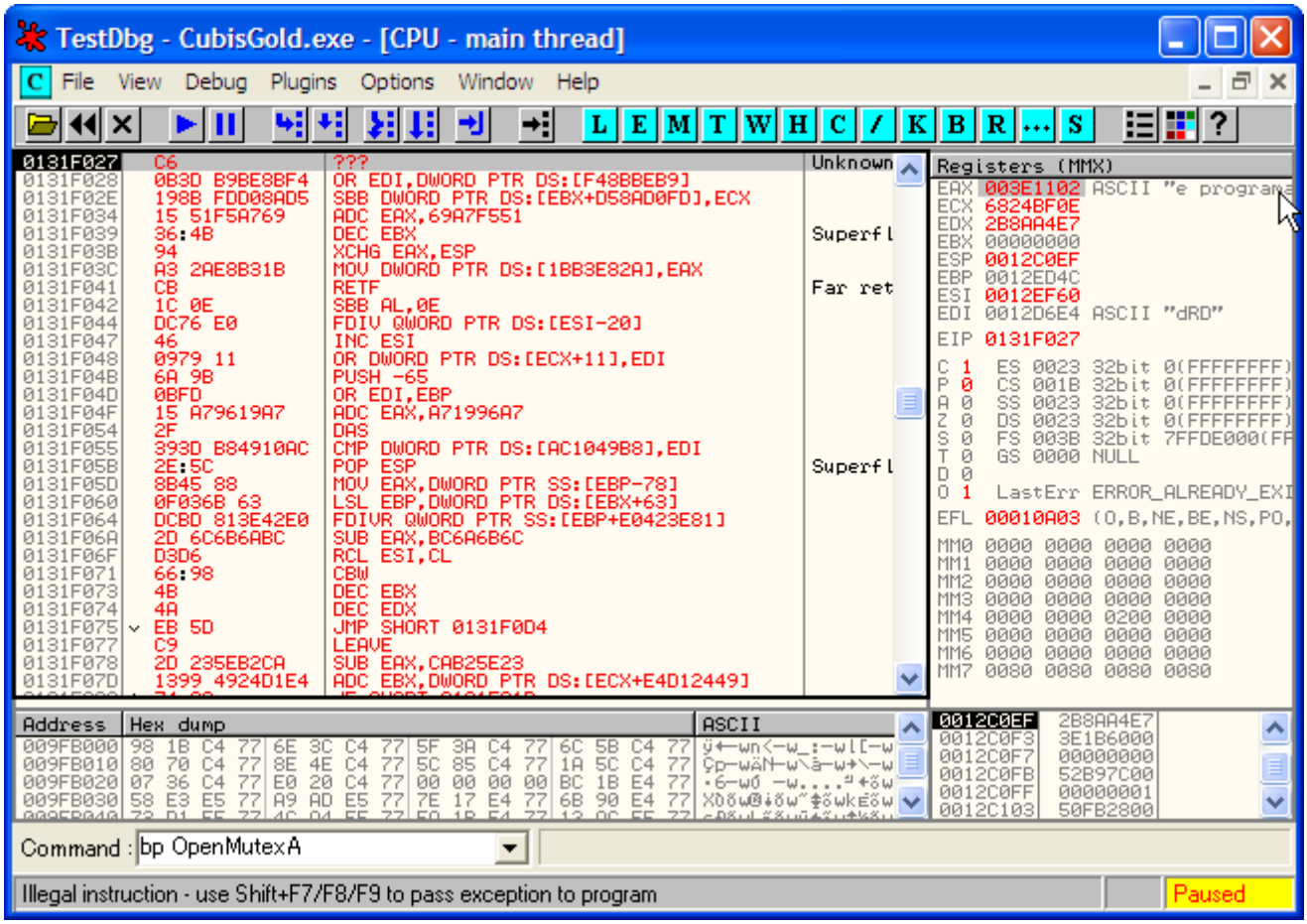
Tenemos que ***ECX==01308195*** que es el valor redirigido de la ***Api*** supuestamente ***FreeLibrary*** como vemos en ***EDX***, anotamos la dirección:

<u>Salto mágico</u>: 131EAEA

Con estos datos estamos en disposición de dar el golpe mortal a la víctima, reunamos todos los pasos en uno sólo, necesitamos hacerlo todo en la misma ejecución, comenzaremos evitando que se cree el segundo proceso mediante ***OpenMutexA*** como quedó detallado anteriormente, deberemos poner un ***BPHE*** en el ***mágico*** y otro en el ***OEP*** que ya conocemos ***:43A6D9***, cuando Olly pare en el mágico lo nopeamos:

```
0131EAEA   90                NOP
0131EAEB   90                NOP
0131EAEC   0FB785 7CE7FFF    MOVZX EAX,WORD PTR SS:[EBP-1884]
0131EAF3   85C0              TEST EAX,EAX
0131EAF5 v 74 0F             JE SHORT 0131EB06
```

y damos ***RUN*** hasta llegar al ***OEP***, pero no lo hace para aquí:



Si intentamos superar la excepción estamos muertos, esto ocurre por no tener la precaución adecuada ante estas situaciones, deberíamos habremos preparado un script en lugar de manejar y modificar el código del programa conociendo ***Armadillo*** como lo conocemos ya, entonces el siguiente paso será crear el ***script*** no será nada complicado e incluso ya está programado por otros ***crackers*** de la lista, no digo nombres porque creo que es mérito de todos, sería una cosita así de sencilla:

```
====================/ script /====================
start:
eob break
run

break:
cmp eip, 131EAEA
jne exit
mov eip, 131EAEC
jmp start

exit:
ret
====================/ script /====================
```

Con este diminuto script evitamos la redirección de los punteros que quedaran en su sitio ;XDDD ***Utilizaremos*** como punto gatillo el propio salto mágico, ponemos un ***BPHE*** en el sino el script no funcionara y otro en el ***OEP*** para ***dumpear*** con la tabla reparada reiniciamos con ***Ctrl+F9*** damos ***RUN***, (no olvidar lo del ***OpenMutexA***) saldrá la enorme nag que aceptamos y acto seguido caeremos en el mágico, desde aquí mismo ejecutamos el script que nos dejara en el ***OEP*** con la ***IAT*** preparada para que el ***ImpRec*** nos la repare, jeje.. este será el indicador que terminó el script correctamente dejándonos en el ***OEP*** gracias al ***HE*** que pusimos en él sino no pararía aquí:

Aceptamos y si todo ha ido bien estaremos en pleno ***OEP*** con la tabla reparada, el ***Import*** dirá que hay 8 entradas inválidas pero son las ***dwords*** separadoras de ***DLL's*** con lo que haremos ***"Cut Thunks"*** de todas ellas pero aún no tenemos el dumpeado, antes de nada vamos a la ventana de memoria y vemos:

003F0000	00002000				Map	R	R
00400000	00001000	CubisGol			Imag	R	RWE
00401000	00044000	CubisGol	.text		Imag	R	RWE
00445000	00006000	CubisGol	.rdata		Imag	R	RWE
0044B000	00580000	CubisGol	.data		Imag	R	RWE
009CB000	00020000	CubisGol	.text1	code	Imag	R	RWE
009EB000	00010000	CubisGol	.adata	code	Imag	R	RWE
009FB000	00020000	CubisGol	.data1	data,import	Imag	R	RWE
00A1B000	00060000	CubisGol	.pdata		Imag	R	RWE
00A7B000	0029B000	CubisGol	.rsrc	resources	Imag	R	RWE

vale lo de siempre nos ha modificado el header del ejecutable para que no podamos obtener un dumpeado correcto, como ya lo he explicado muchas veces (bueno no tantas, jeje..) sólo diré como repararlo, vamos al ***offset*** 0x3Ch y allí ponemos el ***offset*** de donde se encuentre la cadena ***'PE'*** que está en:

Address	Hex dump	ASCII
004000F8	50 45 00 00 4C 01 0C 00 97 B2 6C 3F 00 00 00 00	PE..L0..ù▒l?....
00400108	00 00 00 00 E0 00 0F 01 0B 01 53 52 00 30 02 00	....α.*0♂0SR.0☻.
00400118	00 90 2F 00 00 00 00 00 EB AF 5E 00 00 B0 5C 00	.É/.....ù»^..▒\.

en este caso está en ***:4000F8*** cuyo ***offset*** es 0F8h este valor debería ir en el ***offset*** 0x3Ch citado antes que pertenece al ***PE Signature*** que no es más que un puntero al ***PE*** del archivo, lo ponemos:

Address	Hex dump	ASCII
0040003C	F8 00 00 00 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C	°...♥▼‖♥.┤.═!╕☺L
0040004C	CD 21 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63	═!This program c
0040005C	61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20	annot be run in
0040006C	44 4F 53 20 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00	DOS mode....$...

Allí pusimos el valor correcto, si lo miramos ahora con ***Alt+M*** veremos todo correcto:

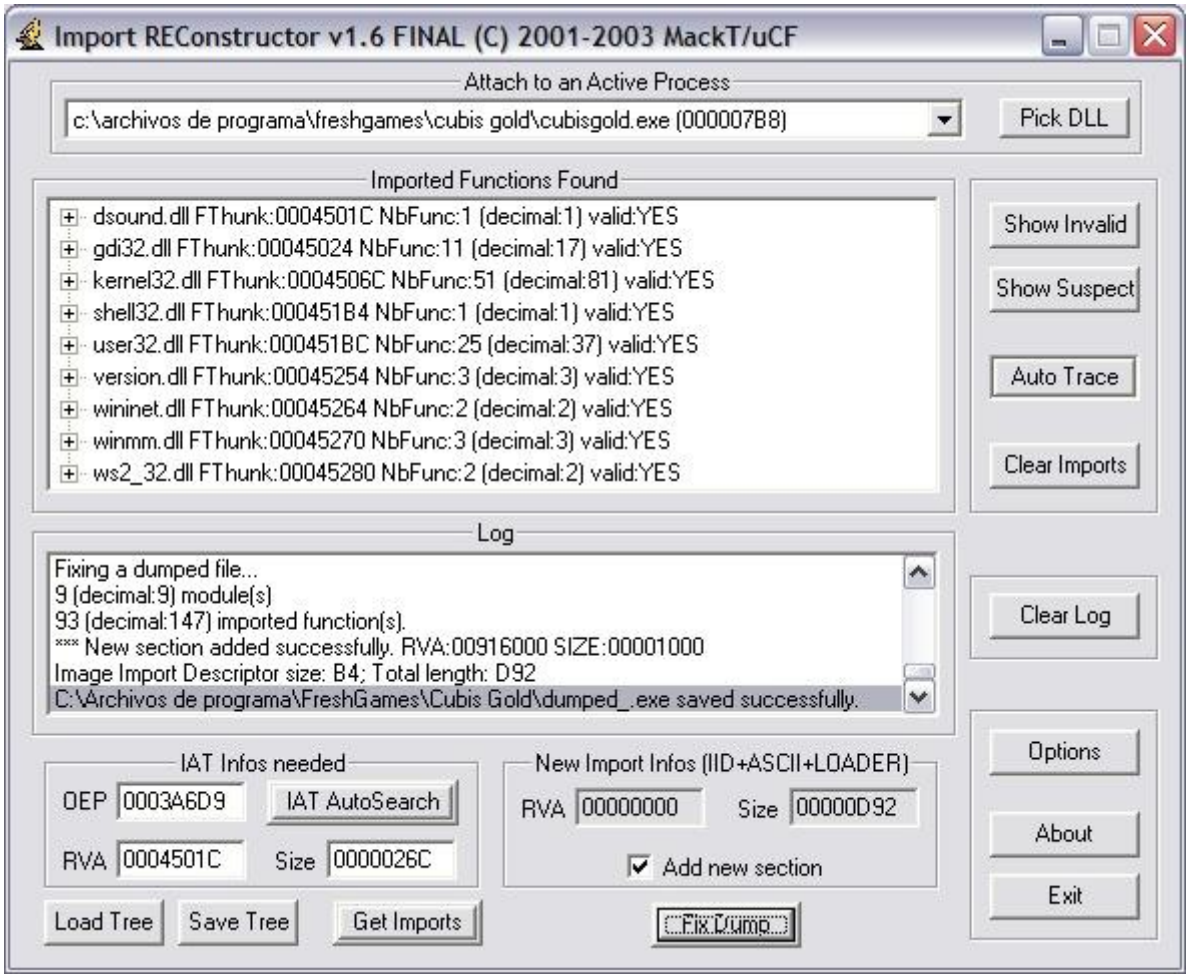
003F0000	00002000				Map	R	R
00400000	00001000	CubisGol		PE header	Imag	R	RWE
00401000	00044000	CubisGol	.text		Imag	R	RWE
00445000	00006000	CubisGol	.rdata		Imag	R	RWE
0044B000	00580000	CubisGol	.data		Imag	R	RWE

queda solucionada la primera parte porque aún queda otra pero nada grave, nos ha cambiado también el número de secciones, nos puso 12 cuando en realidad sólo tiene 8, o sea que nos añadió 4 por todo el morro. cosas del ***Armadillo*** cuando se ver perdido, jejeje... reparamos este valor, nos vamos a la ventana de memoria otra vez y

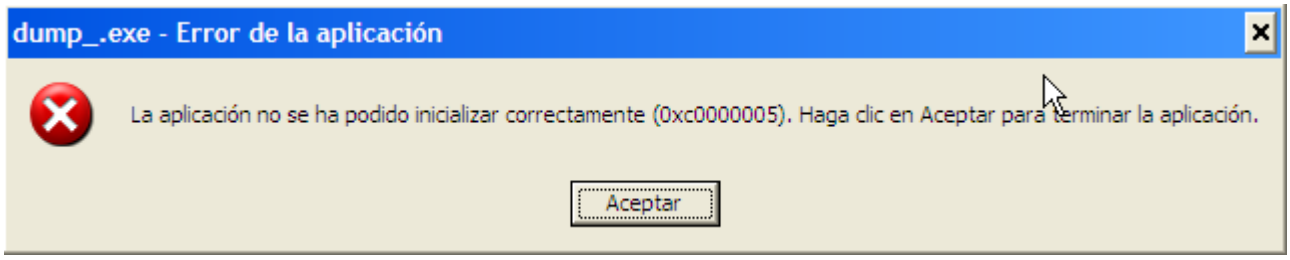
hacemos doble clic sobre la sección de la cabecera, lo ponemos en ***Special View*** para verlo mejor, buscamos el valor ***'NumberOfSections'*** y con el botón derecho picamos para que se abra un menú desplegable donde elegimos ***'Modify Integer'*** y lo cambiamos a 8 como se ve en la siguiente imagen:

```
004000F8   50 45 00 0(  ASCII "PE"        PE signature (PE)
004000FC   4C01         DW 014C           Machine = IMAGE_FILE_MACHINE_I386
004000FE   0800         DW 0008            NumberOfSections = 8
00400100   97B26C3F     DD 3F6CB297        TimeDateStamp = 3F6CB297
00400104   00000000     DD 00000000        PointerToSymbolTable = 0
```

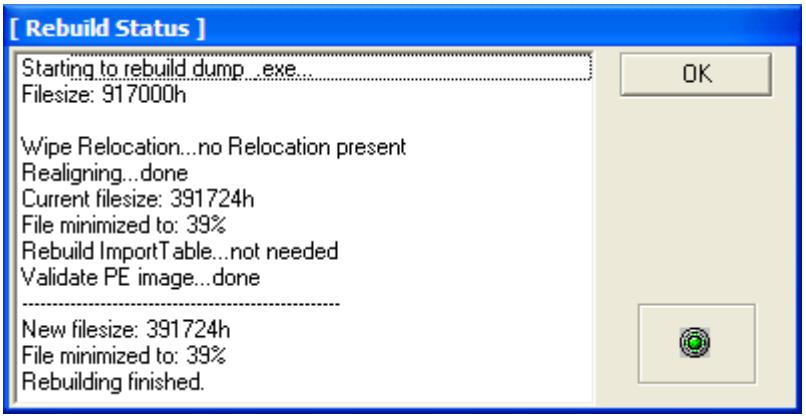
Ahora sí podemos dumpear tranquilamente con el mismo plugin de Olly que a mi me va muy bien, lo configuramos como siempre, destildamos el "***Rebuild Imports***" que no va muy bien digamos y dejamos eso sí tildada la pestaña "***Fix Raw Size***..." pulsamos también el "***Get EIP as OEP***" y dumpeamos, el archivo lo llamaremos dump.exe por ejemplo y en el Import Reconstructor repararemos este, como lo tenemos además preparado de antes sólo debemos picar "***Fix Dump***" y elegirlo...



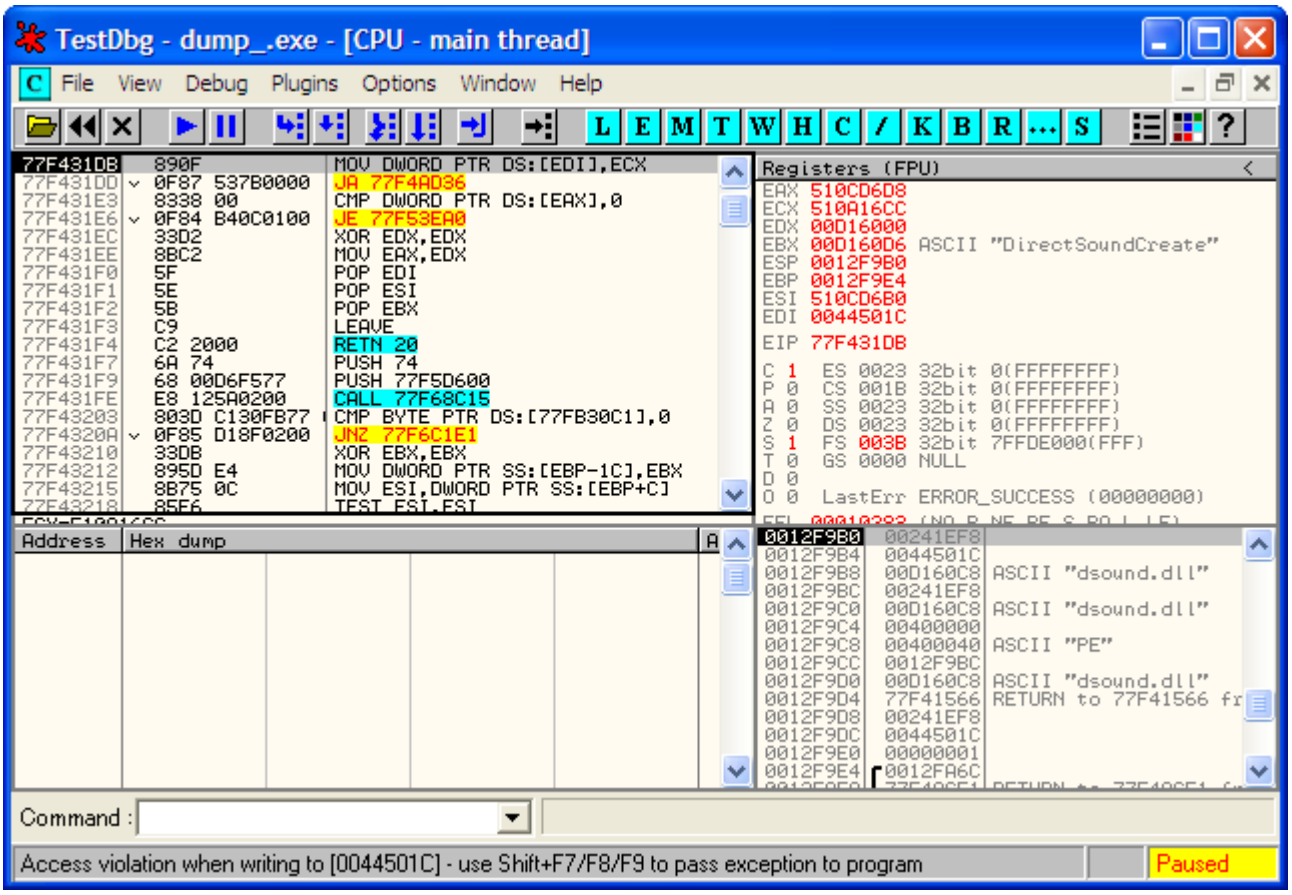
Ya tenemos dump_.exe con la ***IAT*** reparada, jeje.. lo ejecutamos para disfrutaaaaaaaaaar... queeeeeeeeé????



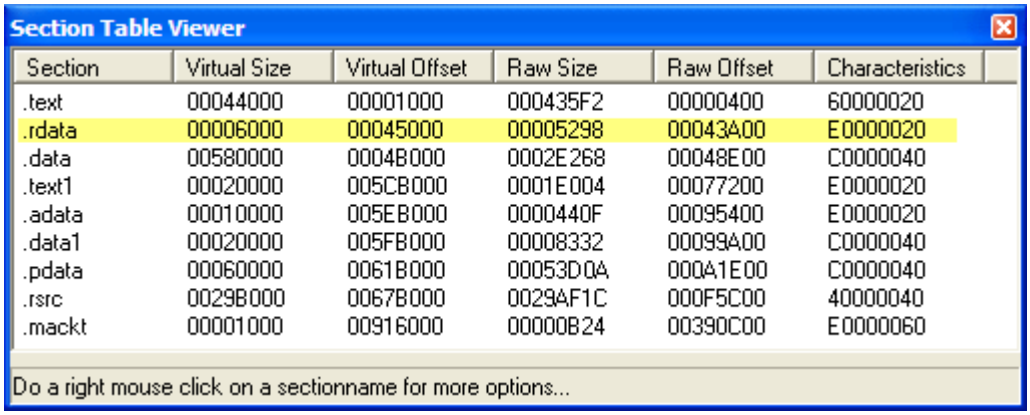
Que es esto? error ***0xC0000005*** no os suena? el típico ***ACCESS_VIOLATION***, esto me suena muuuucho, primero de todo siempre probaremos lo más sencillo y poco a poco lo iremos complicando según veamos si no se repara el asunto, de momento abrimos el ***LordPE*** y arrastramos el dumpeado que acabamos de reparar con el Import (dump_.exe) y lo soltamos justo sobre el ***LordPE*** para que le haga un ***rebuid*** (reconstrucción), vamos que hay que hacerle un arreglo, esto lo leí de un tute de Ricardo donde hacía esto y quedaba reparado el error. Pues nada lo arrastramos y lo soltamos justo encima, aparecerá esta ventanita donde nos informa de lo que le ha hecho:



Cuando termina probamos el dumpeado de nuevo pero tampoco corre ¿¿?? el mismo error por todos lados, nada pues carguémoslo en otro ***Olly*** a ver...



ahí se quedó tirado, si nos fijamos vemos que efectivamente es un ***ACCESS_VIOLATION*** y el motivo, jejeje... está intentando rellenar la ***IAT*** y se encuentra que no tiene permiso para hacerlo, más bien seguramente la sección tendrá las características de sólo lectura (***READ_ONLY***) y por eso peta, probaremos entonces a darle otros privilegios ;-) se los daremos más bien todos con ***PEditor*** por ejemplo que va muy bien:



Section Table Viewer

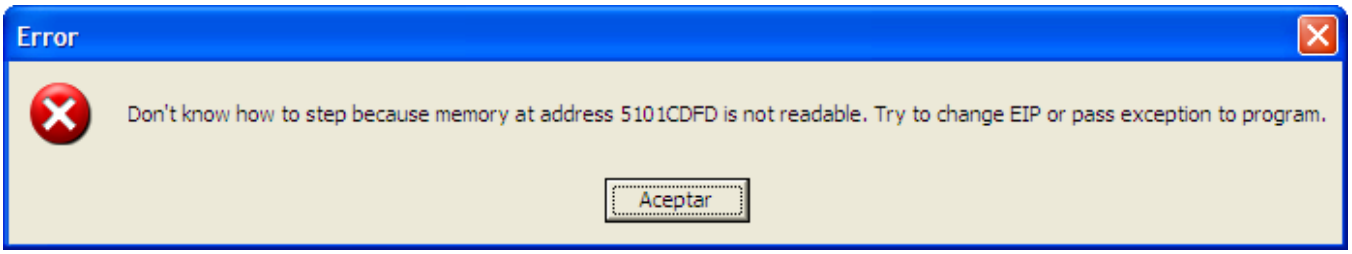
Section	Virtual Size	Virtual Offset	Raw Size	Raw Offset	Characteristics
.text	00044000	00001000	000435F2	00000400	60000020
.rdata	00006000	00045000	00005298	00043A00	E0000020
.data	00580000	0004B000	0002E268	00048E00	C0000040
.text1	00020000	005CB000	0001E004	00077200	E0000020
.adata	00010000	005EB000	0000440F	00095400	E0000020
.data1	00020000	005FB000	00008332	00099A00	C0000040
.pdata	00060000	0061B000	00053D0A	000A1E00	C0000040
.rsrc	0029B000	0067B000	0029AF1C	000F5C00	40000040
.mackt	00001000	00916000	00000B24	00390C00	E0000060

Do a right mouse click on a sectionname for more options...

Cambiamos las características de ***.rdata*** que es la sección donde irá la ***IAT***, abrimos de nuevo el dumpeado ***Olly***:

```
0043A6D9   55              PUSH EBP
0043A6DA   8BEC            MOV EBP,ESP
0043A6DC   6A FF           PUSH -1
0043A6DE   68 E0804400     PUSH dump_ok.004480E0
0043A6E3   68 94F24300     PUSH dump_ok.0043F294
0043A6E8   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0043A6EE   50              PUSH EAX
0043A6EF   64:8925 000000  MOV DWORD PTR FS:[0],ESP
0043A6F6   83EC 58         SUB ESP,58
0043A6F9   53              PUSH EBX
0043A6FA   56              PUSH ESI
0043A6FB   57              PUSH EDI
0043A6FC   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
0043A6FF   FF15 28514400   CALL DWORD PTR DS:[<&kernel32.GetVersio  kernel32.GetVersion
0043A705   33D2            XOR EDX,EDX
```

Ahora sí que lo carga bien, veamos si corre... ***F9*** ... y ...



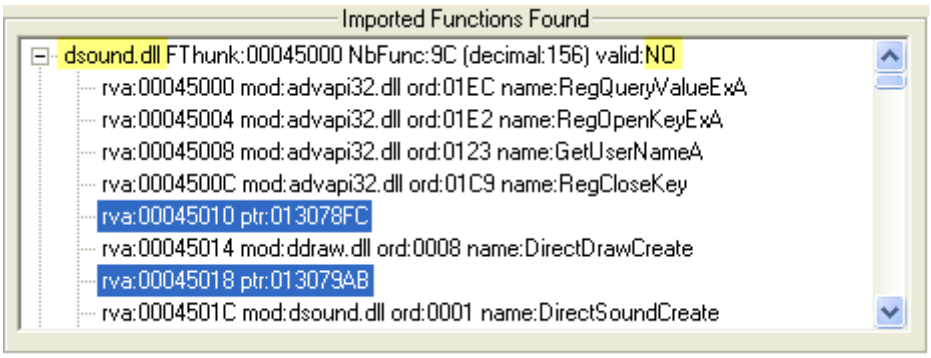
Pues nada no hay manera..!! Tras muchas vueltas di con el error: un puntero de la ***IAT***:

```
00437273   CC              INT3
00437274 - FF25 1C504400   JMP DWORD PTR DS:[<&dsound.DirectSoundC  dsound.DirectSoundCreate
0043727A - FF25 14504400   JMP DWORD PTR DS:[445014]
00437280   E8 05000000     CALL dump_ok.0043728A
00437285 v E9 0D000000     JMP dump_ok.00437297
```

Quedó apuntando a no se sabe donde ¿? en principio creí que era alguna otra trampa de ***Armadillo*** pero no era así, era un error mío al calcular la ***IAT*** le di inicio en ***:44501C*** y realmente empieza en ***:445000***, lo mejor de todo es que este valor lo obtuvimos bien antes, se me colaría al copiarlo al Import, pues nada más trabajo, como no hemos cerrado el otro ***Olly*** con el programa parado en el ***OEP*** este nos servirá, borramos el ***dump_.exe*** de la carpeta y volvemos a reparar el ***dump.exe***, el que no lo tenga detenido en el ***OEP*** deberá repetir la jugada hasta llegar a él con la ***IAT*** reparada y luego ir al Import con estos valores:

Esta vez sí calculamos bien los valores, el inicio en ***:445000*** y el largo es de ***290*** como ya calculamos en la primera etapa, jeje.. parece el Tour, con estos datos ya no nos debe dar más problemas y si los diera podríamos estar casi seguros que de esto no será

Sólo quería mostrar ahora como nos muestra el Import algunos punteros:



lo que remarco en amarillo donde dice ***dsound.dll*** ya se ve que esa ***DLL*** no corresponde ya que vemos algunas entradas de ***ADVAPI32.dll*** lo que ocurre es que nos dejó algunos valores basura por el medio para estorbar, estos punteros se nos habían quedado antes fuera de la ***IAT*** ¡como iba a funcionar!

Reparamos la tabla de nuevo (respetando los nuevos valores), luego cambiamos las propiedades de ***.rdata*** como antes a ***E0000020*** y probamos el invento, hacemos doble clic sobre el nuevo dumpeado y se ejecuta a la primera sin rechistar, parece ser que ***Armadillo*** no opuso mucha resistencia esta vez, pues soy bastante novato en esto y al ver Arma 3.00a-3.60 me temí algo peor pero luego ya veis que hay ***Armadillos*** mucho más complicados que este,

pero en fin mejor para nosotros, hemos disfrutado viendo caer a este grande una vez más, espero que todos los que lo hagáis escribáis cómo lo hicisteis, eso puede ayudar mucho a los demás para cuando les toque a ellos ;)

Bueno muchachos esto no da para más ya véis que no, supongo que a este programa lo protegieron con el mínimo de opciones de ***Armadillo***, como siempre digo pronto más y mejor, hasta la próxima…

Si necesitáis consultar algo o lo que sea me podéis escribir a estas direcciones (mejor a la primera):

syxe00@yahoo.es
syxe00@hotmail.com